NoSQL y nuevas formas de almacenamiento

Trabajo Práctico · Unidad 7 — Food Store

Integrantes	Ismael Saleme · Sabrina Gimenez · Jeronimo Coronel · Joaquin Godoy
Repositorio	github.com/ismaSaleme12/Proyecto_Food_Store_MongoDB
Motores	PostgreSQL 17 (practica_bd2_tp) · MongoDB 9.0.4 (mongosh 2.14.1)

1. Síntesis

Se migraron 10 pedidos reales de Food Store a una colección documental, conservando los identificadores de origen y los precios históricos. Se ejecutaron las cuatro consultas pedidas en MongoDB y sus equivalentes en PostgreSQL: los resultados coinciden en cantidad y en identificadores. El documento concluye que PostgreSQL debe seguir siendo la fuente de verdad y que MongoDB solo aporta valor como proyección de lectura del pedido completo.

2. Parte 1 · Modelo documental y migración

2.1 Subconjunto y tablas de origen (Consigna 1)

Tabla PostgreSQL	Clave	Rol en el subconjunto	Destino en MongoDB
`pedido`	`id` PK (IDENTITY)	cabecera de la venta	documento raíz (`_id`)
`detalle_pedido`	`(pedido_id, producto_id)` PK compuesta	líneas de la venta	arreglo `lineas[]`
`cliente`	`id` PK, `email` UK	datos del comprador	objeto embebido `cliente{}`
`producto`	`id` PK, FK `categoria_id`	descripción del ítem	embebido por línea
`categoria`	`id` PK, `nombre` UK	rubro del producto	campo `lineas[].categoria`

Intervienen cinco tablas y dos claves foráneas encadenadas: `pedido` → `cliente`, `detalle_pedido` → `pedido`, `detalle_pedido` → `producto`, `producto` → `categoria`.

2.2 Esquema de la colección propuesta

```
{
  _id: 1,                          // = pedido.id original (identificador conservado)
  pedido_id: 1,                    // copia explícita, para trazabilidad
  fecha: ISODate("2026-06-28T23:06:30.148Z"),
  forma_pago: "EFECTIVO",          // rol de "estado" (ver 2.3)
  cliente: { cliente_id: 453, nombre: "Cliente 453",
             email: "cliente453@example.com", telefono: "555-000453" },
  lineas: [                        // antes: filas de detalle_pedido
    { producto_id: 1, nombre: "Producto 1", categoria: "Bebidas",
      cantidad: 5, precio_unitario: 1868.53, subtotal: 9342.65 }
  ],
  total: 9342.65,                  // derivado, congelado al momento de la venta
  origen: { motor: "PostgreSQL", base: "practica_bd2_tp", tablas: [...] }
}
```

Campo	Tipo	Origen relacional	Decisión
`_id` / `pedido_id`	entero	`pedido.id`	se conserva el ID original, no se genera ObjectId
`fecha`	date	`pedido.fecha`	`TIMESTAMPTZ` se guarda en UTC
`forma_pago`	string	`pedido.forma_pago` (ENUM)	el dominio del ENUM se replica con `$jsonSchema`
`cliente{}`	objeto	`cliente`	embebido parcial: solo los datos usados al leer el pedido
`lineas[]`	arreglo	`detalle_pedido`	embebido: entidad débil sin vida propia
`lineas[].precio_unitario`	decimal	`detalle_pedido.precio_unitario`	precio histórico, nunca se recalcula
`total`	decimal	derivado	precalculado para evitar `$sum` en cada lectura

2.3 Correspondencia «estado» → `forma_pago`

El modelo real de Food Store **no tiene columna** `estado` en `pedido`. El atributo del dominio que cumple ese rol es `forma_pago` (`EFECTIVO | TARJETA | TRANSFERENCIA`). Se mantiene el nombre real del campo en ambos motores para que la comparación de la Consigna 5 sea sobre el mismo dato, en lugar de inventar un atributo que la base no tiene.

2.4 Proceso de migración (Consigna 2)

La migración se hizo en dos pasos, sin retipear datos a mano:

1. **Extracción desde PostgreSQL** (`consultas_postgresql.sql`, Bloque 1): una consulta con `jsonb_agg` + `jsonb_build_object` arma cada pedido completo como un documento JSON, agregando las líneas con `jsonb_agg` y calculando `total` con `sum()`.
2. **Carga en MongoDB** (`consultas_mongodb.js`, Bloque 1): los documentos se insertan con `insertMany()`. La alternativa equivalente es volcar la salida del paso 1 a `datos_migracion.json` y cargarla con `mongoimport --jsonArray`.

Evidencia de ejecución:

```
Documentos insertados: 10
IDs migrados: [1, 2, 3, 4, 5, 6, 7, 50001, 100001, 150001]
Total facturado en la coleccion: $85928
```

Se eligieron 10 pedidos (el mínimo pedido era 5) que cubren las tres formas de pago y que incluyen cuatro pedidos que comparten el producto 1, para que la consulta c) tenga más de un resultado y la comparación no sea trivial.

> **Detalle de migración:** PostgreSQL guarda `TIMESTAMPTZ` con offset `-03`; MongoDB lo normaliza a UTC. El pedido 50001, `2026-05-03 21:32:56-03`, queda como `2026-05-04T00:32:56Z`. Es el mismo instante: solo cambia la zona de representación.

2.5 Reglas de integridad conservadas (Consigna 3)

Regla de Food Store	Cómo se conserva en MongoDB	Verificación
R1 — Solo productos activos en el detalle	Se valida en la extracción (el `JOIN` con `producto` filtra `activo`); en runtime no es exigible por el motor porque `producto` vive en otra colección	Bloque 5 del SQL: `lineas_inactivas = 0` en los 10 pedidos
R2 — Todo pedido tiene al menos una línea	Pasa a ser **estructural**: `lineas` es un arreglo con `minItems: 1` en el validador `$jsonSchema`	Prueba negativa: un documento con `lineas: []` es rechazado por el motor

Qué se embebió y por qué. Se embebieron `lineas[]` y `cliente{}`, y se dejaron afuera el catálogo de productos, las categorías y el stock:

— `detalle_pedido` es una **entidad débil**: no existe una línea sin pedido, siempre se lee junto con la cabecera y su cardinalidad está acotada por pedido. Embeberla elimina el `JOIN` y permite que la escritura del pedido completo sea **atómica en un solo documento**, sin transacción multi-documento.

— `cliente` se embebe **parcialmente** (nombre, email, teléfono) porque es el dato que se muestra al leer el pedido; se conserva `cliente_id` para poder volver a la entidad.

— `producto` se embebe **solo como descripción histórica** (nombre y categoría) junto al `precio_unitario` congelado. El catálogo mutable —precio actual, stock, `activo`— queda fuera: es dato compartido por miles de pedidos y duplicarlo generaría inconsistencia.

2.6 Hallazgo sobre el dataset

En la base real, `pedido` y `detalle_pedido` tienen 200.000 filas cada una: **la carga masiva del TP3 generó exactamente una línea por pedido**, de modo que la relación N:M quedó degenerada en 1:1. El modelo documental soporta N líneas (el arreglo y la consulta c) lo demuestran), pero conviene señalarlo: cualquier conclusión sobre «muchas líneas por pedido» no está respaldada por los datos actuales.

3. Parte 2 · Consultas y verificación

3.1 Consultas ejecutadas en MongoDB (Consigna 4)

```
// a) Todos los pedidos con identificador y estado
db.pedidos.find({}, { _id: 1, fecha: 1, forma_pago: 1, total: 1 }).sort({ _id: 1 });

// b) Filtro por un estado existente en los datos
db.pedidos.find({ forma_pago: "TARJETA" }, { _id: 1, forma_pago: 1 });

// c) Pedidos que contienen un producto dentro del arreglo de líneas
db.pedidos.find({ "lineas.producto_id": 1 }, { _id: 1, lineas: { $elemMatch: { producto_id: 1 } } });

// d) Pedidos con total superior a un umbral
db.pedidos.find({ total: { $gt: 10000 } }, { _id: 1, total: 1 }).sort({ total: -1 });
```

Salida real:

```
===== a) Todos los pedidos con su identificador y estado =====
1 | 2026-06-28 | EFECTIVO | $9342.65        6 | 2026-06-25 | EFECTIVO | $4971.38
2 | 2026-06-22 | TARJETA | $19389.36        7 | 2025-09-13 | TRANSFERENCIA | $12817.15
3 | 2026-04-24 | TARJETA | $14254.05        50001 | 2026-05-04 | TARJETA | $1868.53
4 | 2026-06-12 | TRANSFERENCIA | $4609.02   100001 | 2025-12-28 | TRANSFERENCIA | $7474.12
5 | 2026-08-25 | TARJETA | $5596.15         150001 | 2026-03-09 | TRANSFERENCIA | $5605.59

===== b) Pedidos con estado = TARJETA =====
2 | $19389.36    3 | $14254.05    5 | $5596.15    50001 | $1868.53
Cantidad: 4   IDs: [2, 3, 5, 50001]

===== c) Pedidos que contienen el producto 1 =====
1 | Producto 1 | cant 5 x $1868.53        100001 | Producto 1 | cant 4 x $1868.53
50001 | Producto 1 | cant 1 x $1868.53    150001 | Producto 1 | cant 3 x $1868.53
Cantidad: 4   IDs: [1, 50001, 100001, 150001]

===== d) Pedidos con total > 10000 =====
2 | $19389.36    3 | $14254.05    7 | $12817.15
Cantidad: 3   IDs: [2, 3, 7]
```

Justificación del umbral de d). Se eligió **$10.000** con datos, no a ojo: sobre los 200.000 pedidos de la base el total promedio es **$8.233,91**, la mediana **$6.774,93** y el p90 **$17.210,54**. Un corte en $10.000 deja por encima al **31,6 %** de los pedidos (63.178 de 200.000) —el tercio de mayor valor— y en la muestra migrada selecciona 3 de 10, proporción consistente con la de la base completa.

3.2 Equivalentes en PostgreSQL (Consigna 5)

```
-- b) estado
SELECT p.id, p.forma_pago, p.fecha
FROM pedido p
WHERE p.forma_pago = 'TARJETA'
  AND p.id = ANY (ARRAY[1,2,3,4,5,6,7,50001,100001,150001])
ORDER BY p.id;

-- c) producto dentro del detalle
SELECT DISTINCT p.id, pr.nombre, d.cantidad, d.precio_unitario
FROM pedido p
JOIN detalle_pedido d  ON d.pedido_id  = p.id
JOIN producto       pr ON pr.id        = d.producto_id
WHERE d.producto_id = 1
  AND p.id = ANY (ARRAY[1,2,3,4,5,6,7,50001,100001,150001])
ORDER BY p.id;
```

La comparación se restringe a los 10 pedidos migrados. Comparar contra los 200.000 pedidos de la base no sería equivalente: la consigna pide el **mismo conjunto de datos** en ambos motores.

3.3 Tabla de verificación

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs
Estado seleccionado: `TARJETA`	4	4	**Sí** — `{2, 3, 5, 50001}`
Producto seleccionado: `Producto 1` (id 1)	4	4	**Sí** — `{1, 50001, 100001, 150001}`

Salida cruda de `psql` (Bloque 4):

```
     consulta     | cantidad |           ids
------------------+----------+-------------------------
 Estado = TARJETA |        4 | {2,3,5,50001}
 Producto = 1     |        4 | {1,50001,100001,150001}
```

3.4 Diferencias detectadas y correcciones

No hubo diferencias de resultado. Sí hay diferencias de forma que conviene documentar:

1. **Consulta c) escrita de forma ingenua devuelve 0 filas.** `find({ lineas: { producto_id: 1 } })` compara el arreglo completo por igualdad exacta de documento, no «contiene». Se corrigió a notación de punto `"lineas.producto_id"` (o `$elemMatch`), que es la forma que usa el índice multikey `idx_lineas_producto`. En PostgreSQL el error equivalente sería olvidar el `DISTINCT` y devolver el pedido repetido por cada línea coincidente.
2. **Zona horaria:** PostgreSQL devuelve `2026-05-03 21:32:56-03`, MongoDB `2026-05-04T00:32:56Z`. Es el mismo instante; se documenta para que la diferencia no se lea como un error de carga.
3. **El JOIN desaparece:** en MongoDB la lectura del pedido completo es un único `find` por `_id`; en PostgreSQL requiere unir `pedido` + `cliente` + `detalle_pedido` + `producto`.

4. Parte 3 · Evaluación de la tecnología e IA

4.1 Consulta realizada a la IA (Consigna 6)

> «Estamos modelando los pedidos de Food Store. El modelo actual es PostgreSQL con `pedido`, `detalle_pedido`, `cliente` y `producto`. El acceso más frecuente es al pedido completo con todas sus líneas, y necesitamos conservar la coherencia de cantidades y precios históricos. ¿Conviene migrar ese subconjunto a MongoDB o mantener PostgreSQL? Compará el almacenamiento y la consulta.»

4.2 Contraste de dos afirmaciones (Consigna 7)

Afirmación 1 — «MongoDB no soporta transacciones ACID, así que no garantiza que un pedido y sus líneas se guarden juntos».

Contraste: es falsa tal como está formulada. MongoDB soporta transacciones ACID multi-documento desde la versión 4.0 en replica sets y 4.2 en sharded clusters (mongodb.com/docs/manual/core/transactions). Pero el argumento además es irrelevante para nuestro modelo: al embeber las líneas, «pedido + líneas» es **un solo documento**, y toda escritura de un documento es atómica por diseño (Write Operations Atomicity). La atomicidad que en PostgreSQL exigía el trigger diferido `trg_verificar_pedido` acá es consecuencia de la estructura. *Verificamos en el laboratorio:* el `insertMany` de 10 documentos se aplicó completo y la prueba negativa con `lineas: []` fue rechazada.

Afirmación 2 — «MongoDB no valida esquema ni restricciones, así que se pierden los CHECK y la integridad del modelo relacional».

Contraste: es parcialmente falsa. MongoDB permite validación declarativa con `$jsonSchema` y `validationAction: "error"` (Schema Validation), que replica `required`, tipos, dominios de ENUM y rangos (`cantidad >= 1`, `precio_unitario >= 0`). *Verificamos en el laboratorio:* el validador rechazó tanto un pedido sin líneas como una línea con `cantidad: 0`. **Donde la afirmación sí acierta** es en la integridad referencial: no hay claves foráneas entre colecciones, así que `producto_id` puede apuntar a un producto inexistente y la Regla 1 (producto activo) no puede sostenerse desde el motor — solo desde la aplicación o un proceso de verificación periódico.

4.3 Conclusión (≤ 200 palabras)

Para el subconjunto pedidos–detalle, Food Store debería **mantener PostgreSQL como sistema de registro** y evaluar MongoDB únicamente como réplica de lectura. Tres criterios técnicos sostienen la decisión. Primero, **integridad referencial**: las líneas dependen de `producto` y `cliente`, y PostgreSQL lo garantiza con claves foráneas que MongoDB no puede imponer entre colecciones; nuestro validador rechaza un pedido sin líneas, pero no puede verificar que el producto exista ni que siga activo (Regla 1). Segundo, **patrón de acceso**: el documento embebido gana cuando se lee el pedido completo —una lectura contra un JOIN de cuatro tablas—, pero el catálogo, el stock y los reportes agregados siguen siendo naturalmente relacionales. Tercero, **volumen y escritura**: 200.000 pedidos con una línea cada uno no justifican cambiar de motor, y el modelo documental quedó degenerado porque la carga masiva produjo una relación 1:1. La combinación razonable es PostgreSQL como fuente de verdad y MongoDB como proyección desnormalizada del pedido completo, alimentada por el mismo proceso que hoy genera los documentos.

5. Anexo · Archivos de la entrega

Archivo	Contenido
`consultas_mongodb.js`	creación de la colección, `insertMany` de los 10 pedidos, validador `$jsonSchema`, índices, consultas a–d y pruebas negativas
`consultas_postgresql.sql`	consulta de extracción (`jsonb_agg`), equivalentes de b) y c), tabla de verificación y control de las Reglas 1 y 2
`datos_migracion.json`	los 10 documentos migrados, en JSON, como respaldo de la carga
`salida_mongodb.txt`	salida completa de `mongosh --file consultas_mongodb.js`
`salida_postgresql.txt`	salida completa de `psql -f consultas_postgresql.sql`
`informe_u7_nosql.md` / `.pdf`	este informe

Reproducción:

```
mongosh "mongodb://127.0.0.1:27017" --file consultas_mongodb.js
psql -U postgres -h localhost -d practica_bd2_tp -f consultas_postgresql.sql
```